

Save a copy of the program we've been working on as *python\_repos\_visual.py*, then modify it so it reads as follows:

```
python_repos
_visual.py

import requests
import plotly.express as px

# Make an API call and check the response.
url = "https://api.github.com/search/repositories"
url += "?q=language:python+sort:stars+stars:>10000"

headers = {"Accept": "application/vnd.github.v3+json"}
r = requests.get(url, headers=headers)
❶ print(f"Status code: {r.status_code}")

# Process overall results.
response_dict = r.json()
❷ print(f"Complete results: {not response_dict['incomplete_results']}")

# Process repository information.
repo_dicts = response_dict['items']
❸ repo_names, stars = [], []
for repo_dict in repo_dicts:
    repo_names.append(repo_dict['name'])
    stars.append(repo_dict['stargazers_count'])

# Make visualization.
❹ fig = px.bar(x=repo_names, y=stars)
fig.show()
```

We import Plotly Express and then make the API call as we have been doing. We continue to print the status of the API call response so we'll know if there is a problem ❶. When we process the overall results, we continue to print the message confirming that we got a complete set of results ❷. We remove the rest of the `print()` calls because we're no longer in the exploratory phase; we know we have the data we want.

We then create two empty lists ❸ to store the data we'll include in the initial chart. We'll need the name of each project to label the bars (`repo_names`) and the number of stars to determine the height of the bars (`stars`). In the loop, we append the name of each project and the number of stars it has to these lists.

We make the initial visualization with just two lines of code ❹. This is consistent with Plotly Express's philosophy that you should be able to see your visualization as quickly as possible before refining its appearance. Here we use the `px.bar()` function to create a bar chart. We pass the list `repo_names` as the `x` argument and `stars` as the `y` argument.

Figure 17-1 shows the resulting chart. We can see that the first few projects are significantly more popular than the rest, but all of them are important projects in the Python ecosystem.